GUÍA PARA LA DEFENSA

Value Iteration optimizado para FrozenLake 8x8

Cómo explicar el algoritmo, justificar gamma = 0.995 y entender qué valor se envía a Kaggle

IDEA CENTRAL

Value Iteration genera la política; la evaluación exacta a 200 pasos selecciona el gamma; y la simulación de 1000 episodios produce el valor que se escribe en rewards.csv.

El proyecto en una mirada

- Entorno: FrozenLake8x8-v1, con 64 estados y 4 acciones.
- Dinámica: estocástica, porque is_slippery=True.
- Algoritmo: Dynamic Programming - Value Iteration.
- Ventaja clave: el modelo exacto de transición está disponible en env.unwrapped.P.
- Objetivo real: maximizar la probabilidad de llegar a la meta antes de 200 pasos.
- Mejor configuración: gamma = 0.995, con éxito exacto aproximado de 0.893568.

Mapa mental

1. Modelo conocido	2. Value Iteration	3. Comparar gammas
P contiene probabilidades, siguientes estados, recompensas y terminales.	Bellman actualiza V hasta que el cambio máximo sea menor que 1e-12.	Cada política se mide exactamente bajo el límite real de 200 pasos.
4. Elegir gamma	**5. Simular**	**6. Exportar**
Se selecciona 0.995 porque alcanza la mayor probabilidad dentro del horizonte.	Se ejecutan 1000 episodios con semillas fijas para obtener el reward promedio.	El promedio se repite en las dos filas exigidas de rewards.csv.

1. Qué problema se está resolviendo

FrozenLake8x8 es un Proceso de Decisión de Markov. El agente ocupa una de las 64 celdas y elige una de cuatro acciones: izquierda, abajo, derecha o arriba. Como el suelo es resbaladizo, la acción elegida no determina por completo el movimiento final.

La recompensa es 1 únicamente al llegar a la meta. Caer en un agujero o no llegar dentro del límite produce recompensa 0. Cada episodio termina, como máximo, después de 200 pasos.

Por qué Dynamic Programming

> *Se usa Value Iteration porque el entorno permite acceder legalmente al modelo exacto env.unwrapped.P. Al conocer las probabilidades de transición, las expectativas se calculan directamente, sin aprenderlas mediante episodios muestreados.*

Parámetro	Valor	Qué significa
Entorno	FrozenLake8x8-v1	Cuadrícula de 8 x 8
Estados	64	Una celda por estado
Acciones	4	Left, Down, Right, Up
Slippery	True	Transiciones estocásticas
Máximo de pasos	200	Horizonte real de cada episodio
Modelo	env.unwrapped.P	Probabilidades exactas de transición
Algoritmo	Value Iteration	Dynamic Programming
Gamma elegido	0.995	Factor de descuento
Tolerancia	1e-12	Criterio de convergencia
Iteraciones reales	1103	Para gamma 0.995
Episodios finales	1000	Evaluación empírica

2. Cómo funciona Value Iteration

La función V(s) estima qué tan conveniente es estar en el estado s si, desde allí, se actúa de forma óptima. En cada iteración se calcula el valor esperado de todas las acciones y se conserva el mayor.

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' \mid s,a) \left[R(s,a,s') + \gamma \cdot V_k(s') \right]$$

Lectura de la ecuación

- $P(s' \mid s,a)$: probabilidad de terminar en s' al ejecutar a desde s.
- $R(s,a,s')$: recompensa inmediata de esa transición.
- γ: cuánto pesa el valor futuro.
- $\max_a$: se escoge la acción con mayor valor esperado.

Ejemplo intuitivo con una acción resbaladiza

Si el agente intenta moverse a la derecha, el modelo puede repartir la probabilidad entre tres resultados: arriba, derecha y abajo. De forma simplificada:

Resultado posible	Probabilidad	Aporte al valor
Estado superior	1/3	$(1/3) \cdot [r + \gamma V(s1)]$
Estado derecho	1/3	$(1/3) \cdot [r + \gamma V(s2)]$
Estado inferior	1/3	$(1/3) \cdot [r + \gamma V(s3)]$

Los tres aportes se suman para obtener Q(s, Right). Se repite el cálculo para Left, Down y Up, y se selecciona la acción con el Q más alto.

Estados terminales

Si la transición termina el episodio, se conserva la recompensa inmediata y no se agrega valor futuro:

$$\text{No terminal: } Q(s,a) = r + \gamma V(s') \quad | \quad \text{Terminal: } Q(s,a) = r$$

> **Caer en un agujero: recompensa 0 y termina. Llegar a la meta: recompensa 1 y termina.**

3. Convergencia: por qué aparecen 1103 iteraciones

El programa comienza con V = 0 para los 64 estados. Esto es válido porque las recompensas son no negativas. Primero adquieren valor positivo los estados cercanos a la meta; después ese valor se propaga hacia estados más lejanos.

El algoritmo se detiene cuando:

$$\max_s |V_{nuevo}(s) - V_{anterior}(s)| < 10^{-12}$$

Con gamma = 0.995, el residuo final fue aproximadamente 9.96 x 10^-13. Como es menor que 10^-12, el algoritmo se considera convergido después de 1103 actualizaciones de Bellman.

> **Forma correcta de decirlo: para gamma 0.995 y tolerancia 1e-12, Value Iteration necesitó 1103 iteraciones en este entorno. No significa que siempre necesite 1103.**

Relación entre gamma y velocidad de convergencia

Gamma	Iteraciones	Lectura
0.99	809	Convergencia más rápida
0.995	1103	Configuración seleccionada
0.999	1625	Más lenta
0.9999	1727	Muy cercana a 1
1.0	1739	Sin descuento

En general, cuando gamma se acerca a 1, la información futura conserva más peso y la convergencia puede hacerse más lenta.

Desempate entre acciones

Si dos acciones tienen exactamente el mismo valor, se usa una prioridad determinista: Left, Down, Right, Up. Su finalidad no es mejorar el agente, sino garantizar reproducibilidad y evitar que las semillas de prueba influyan en la política.

4. La decisión clave: por qué gamma = 0.995

No basta con elegir el gamma más cercano a 1. Cada gamma genera una política diferente y optimiza un objetivo descontado distinto. Para compararlas de manera justa, todas se evalúan con el criterio real de la competencia: llegar a la meta dentro de 200 pasos.

Gamma	Iteraciones	P(éxito antes de 200 pasos)
0.99	809	≈ 0.862955
0.995	1103	0.893568385975
0.999	1625	0.885653919320
0.9999	1727	0.885653919320
1.0	1739	0.885653919320

> **Resultado: gamma = 0.995 obtiene la mayor probabilidad exacta de éxito dentro del límite: aproximadamente 89.36%.**

Por qué 0.999 o 1.0 pueden ser peores

Con gamma = 1 no hay descuento, así que la política favorece llegar eventualmente a la meta. Sin embargo, una ruta que funciona bien a horizonte infinito puede tardar demasiado. La competencia corta el episodio en 200 pasos; por eso importa llegar a tiempo, no solo llegar alguna vez.

gamma = 0.995	**gamma = 0.999**
P(éxito ≤ 200) ≈ 0.893568	P(éxito ≤ 200) ≈ 0.885654
Mejor equilibrio entre seguridad y rapidez.	Política ligeramente peor bajo el horizonte real.

> *No seleccioné gamma mirando solo el valor de Value Iteration. Generé una política con cada gamma y evalué todas con el mismo objetivo real: probabilidad de éxito dentro de 200 pasos. Gamma 0.995 fue el mejor bajo ese criterio.*

5. Dos evaluaciones, dos propósitos

El diseño separa la selección de la política de la medición final exigida por la competencia.

Evaluación	Cómo funciona	Para qué se usa
Exacta basada en el modelo	Propaga probabilidades con env.unwrapped.P durante 200 pasos. No tiene ruido de muestreo.	Comparar políticas y escoger gamma.
Empírica de 1000 episodios	Ejecuta realmente el entorno con semillas fijas y cuenta éxitos.	Obtener el reward promedio que se envía.

Evaluación exacta

Para la política seleccionada:

$$P(\text{éxito antes de 200 pasos}) \approx 0.893568385975$$

Este valor no cambia por azar, porque se calcula directamente a partir de las probabilidades del modelo.

Evaluación empírica

Después se simulan exactamente 1000 episodios. El promedio es:

$$\text{reward promedio} = \text{episodios exitosos} / 1000$$

Si hay 901 éxitos, el promedio es 901/1000 = 0.901. Puede diferir de 0.893568 porque 1000 episodios constituyen una muestra aleatoria.

> **La evaluación exacta elige la mejor política sin ruido estadístico. La simulación final cumple el procedimiento solicitado por la competencia.**

Sobre las semillas

Se utilizan las semillas 0, 1, 2, ..., 999 para reproducibilidad local. La política no se selecciona con esas semillas; solo se usan en la evaluación empírica final.

> *La descripción menciona semillas fijas, pero no especifica cuáles. Por eso uso 0 a 999 para reproducibilidad. No afirmo que sean las semillas oficiales salvo confirmación del profesor.*

6. Qué se envía a Kaggle

El archivo rewards.csv contiene un encabezado y dos filas. Ambas filas repiten el mismo reward promedio obtenido en los 1000 episodios.

```
Id,Expected
1,0.901000
2,0.901000
```

No se envían los dos mejores gammas, las mejores semillas, la probabilidad exacta ni el Absolute Error. Se envía únicamente el promedio empírico, repetido en las dos filas requeridas.

Kaggle calcula después:

$$AE = |Expected - 1|$$

Por ejemplo, Expected = 0.901 produce AE = |0.901 - 1| = 0.099.

¿Por qué no Monte Carlo para aprender?

Monte Carlo aprendería valores mediante episodios muestreados. Aquí el modelo exacto ya está disponible, así que Dynamic Programming calcula directamente las expectativas y evita el ruido de estimación durante la construcción de la política.

> *Monte Carlo no es malo en general. En este problema, Value Iteration es especialmente conveniente porque el modelo de transición ya se conoce.*

Cadena completa del proyecto

1	**Leer P**	Conocer exactamente las transiciones
2	**Ejecutar Value Iteration**	Obtener una política por cada gamma
3	**Evaluar a 200 pasos**	Comparar todas con el objetivo real
4	**Elegir 0.995**	Mayor probabilidad exacta de éxito
5	**Simular 1000 episodios**	Calcular el reward promedio
6	**Crear rewards.csv**	Repetir el promedio en Id 1 e Id 2

7. Preguntas probables del profesor

¿Por qué usaste Value Iteration?

Porque el modelo exacto de transición está disponible en env.unwrapped.P. Puedo calcular expectativas directamente y no necesito aprenderlas mediante simulación.

¿Por qué gamma 0.995 y no 0.999?

Porque todas las políticas se compararon bajo el mismo horizonte de 200 pasos. Gamma 0.995 alcanzó aproximadamente 0.89357, frente a 0.88565 de gamma 0.999.

¿Por qué no gamma = 1?

Gamma 1 favorece llegar eventualmente, pero la competencia impone 200 pasos. Bajo ese límite su política obtuvo aproximadamente 0.88565, menos que gamma 0.995.

¿Por qué 1103 iteraciones?

Para gamma 0.995 y tolerancia 1e-12, el cambio máximo cayó por debajo del umbral después de 1103 actualizaciones. No es una constante universal.

¿Por qué inicializar V en cero?

Las recompensas son no negativas. El valor positivo nace cerca de la meta y se propaga hasta estabilizarse.

¿Por qué dos evaluaciones?

La exacta selecciona la política sin ruido; la empírica de 1000 episodios genera el promedio que exige la competencia.

¿Qué pasa en un estado terminal?

Se conserva la recompensa inmediata y no se suma gamma V(s'). Un agujero da 0; la meta da 1.

¿Qué papel tienen las semillas?

Solo hacen reproducible la evaluación empírica. No se usan para seleccionar gamma ni desempatar acciones.

8. Respuesta corta para memorizar

> *Elegí Dynamic Programming mediante Value Iteration porque la competencia permite usar el modelo exacto env.unwrapped.P. FrozenLake8x8 tiene 64 estados, cuatro acciones y transiciones estocásticas por is_slippery=True. Inicializo los valores en cero y aplico repetidamente la ecuación de Bellman, calculando el valor esperado de cada acción con las probabilidades exactas del entorno. El algoritmo se detiene cuando la diferencia máxima entre iteraciones es menor que 1e-12.*
>
> *Probé gamma 0.99, 0.995, 0.999, 0.9999 y 1.0. Como cada episodio tiene un máximo de 200 pasos, evalué exactamente cada política bajo ese horizonte. La mejor fue gamma 0.995, con una probabilidad de éxito aproximada de 0.89357. Esa configuración convergió en 1103 iteraciones.*
>
> *Finalmente ejecuté la política durante 1000 episodios con semillas fijas. El reward promedio es el valor que escribo en ambas filas de rewards.csv. Kaggle calcula automáticamente el Absolute Error.*

Si te piden las dos mejores configuraciones

> *La mejor fue gamma 0.995, con aproximadamente 0.893568 de éxito exacto dentro de 200 pasos. La segunda fue gamma 0.999, con aproximadamente 0.885654. Gamma 0.9999 y gamma 1.0 produjeron prácticamente el mismo rendimiento bajo ese criterio.*

> **FRASE FINAL**
>
> **Value Iteration genera la política, la evaluación exacta a 200 pasos selecciona el gamma, y la simulación de 1000 episodios genera el valor que se envía a Kaggle.**

Checklist antes de la defensa

- Puedo explicar qué contiene env.unwrapped.P.
- Entiendo la ecuación de Bellman sin memorizar símbolos aislados.
- Puedo justificar gamma 0.995 usando el horizonte de 200 pasos.
- Distingo evaluación exacta de evaluación empírica.
- Sé por qué rewards.csv repite el mismo promedio dos veces.
- No afirmo que 0..999 sean semillas oficiales sin confirmación.